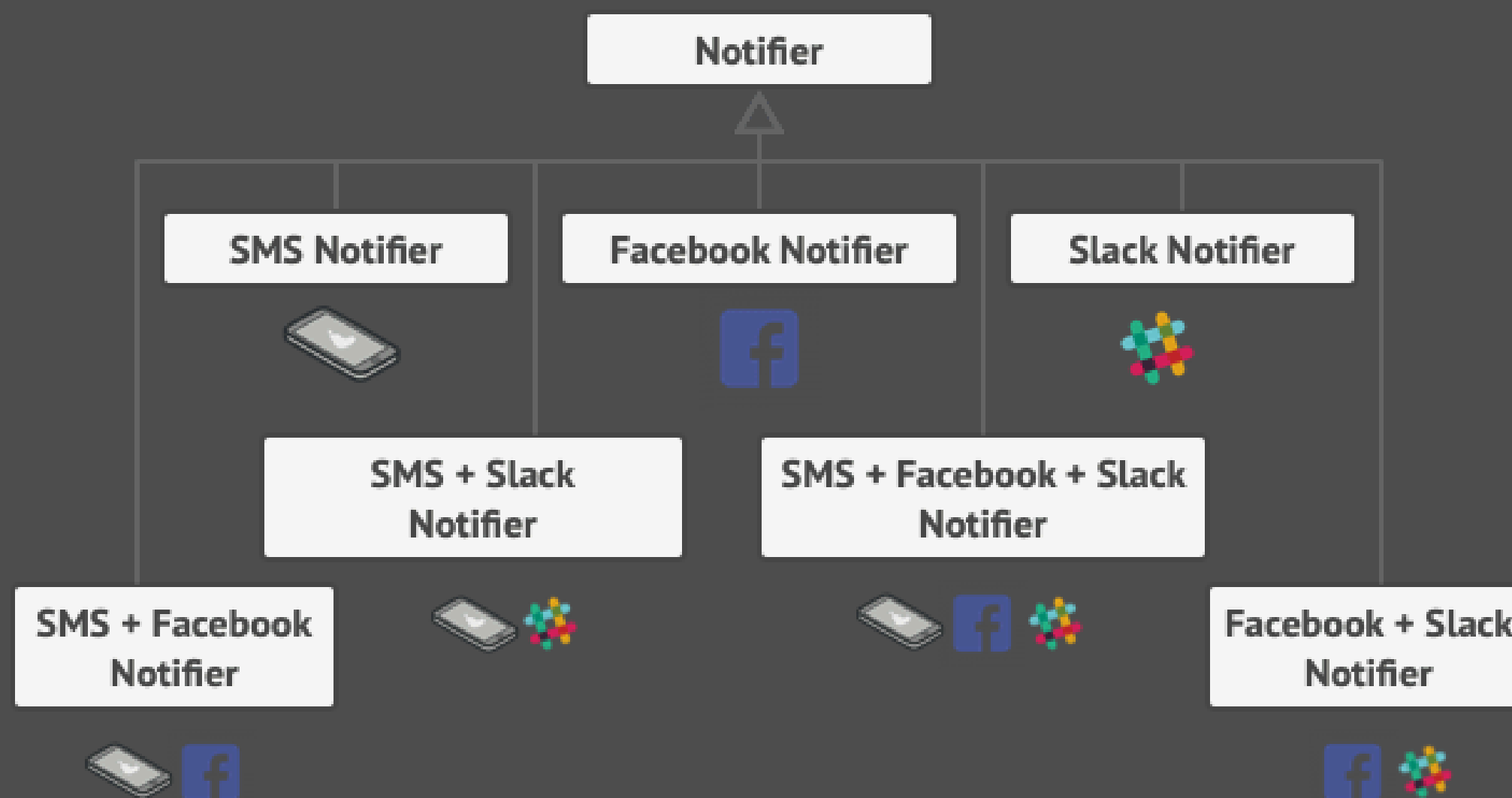


Design Patterns Decorator

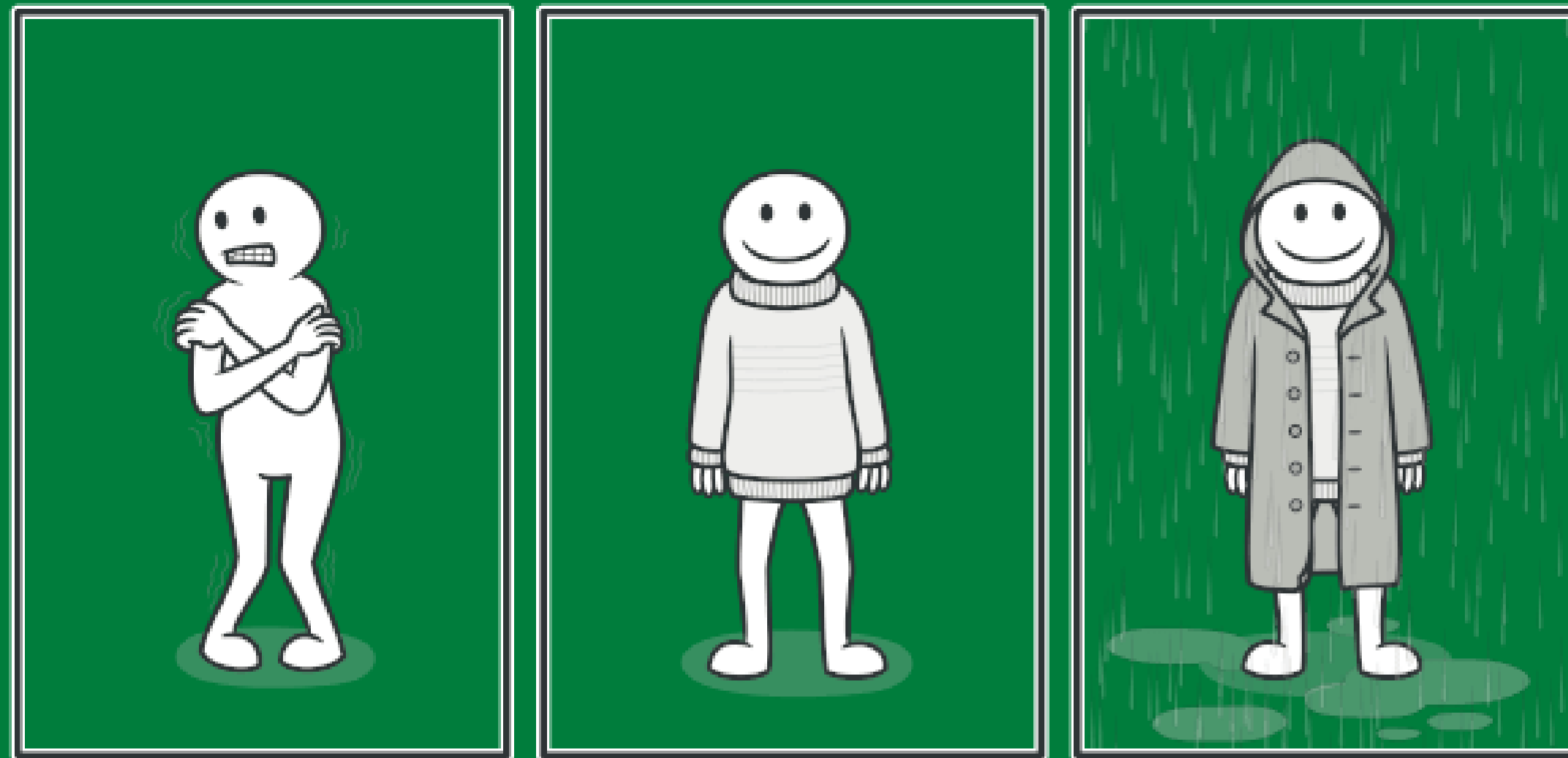
Emilly Aquino





Decorator

É um padrão estrutural do catálogo GoF (Gang of Four), ele permite que um objeto seja modificado, com novos comportamentos sem alterar o comportamento do objeto original



O decorator resolve problemas que a herança gera em determinados momentos, em que classes precisam ser muito especificadas e em detalhes diferentes gerando excessivas subclasses.

- Evitar a explosão de subclasses para prover todas as combinações de responsabilidades
- Acrescentar responsabilidades a um objeto dinamicamente
- Prover alternativa flexível ao uso de subclasses para se estender a funcionalidade de uma classe
- Pode-se utilizar decorator quando a herança seria uma boa alternativa mas a definição da classe está escondida ou não disponível para herança
- Pode usar um ou mais decoradores para englobar um objeto

Como é demonstrado?

- ConcreteComponent, classe que contém os comportamentos originais;
- Component, interface que sua classe original implementa;
- ConcreteDecorator, classe que utiliza o Component e incluir novos comportamentos
- Decorator, interface que sua ConcreteDecorator implementa;

```

interface Cafe {
    double getCusto();
    String getDescricao();
}

class CafeSimples implements Cafe {
    public double getCusto() { return 5.0; }
    public String getDescricao() { return "Café"; }
}

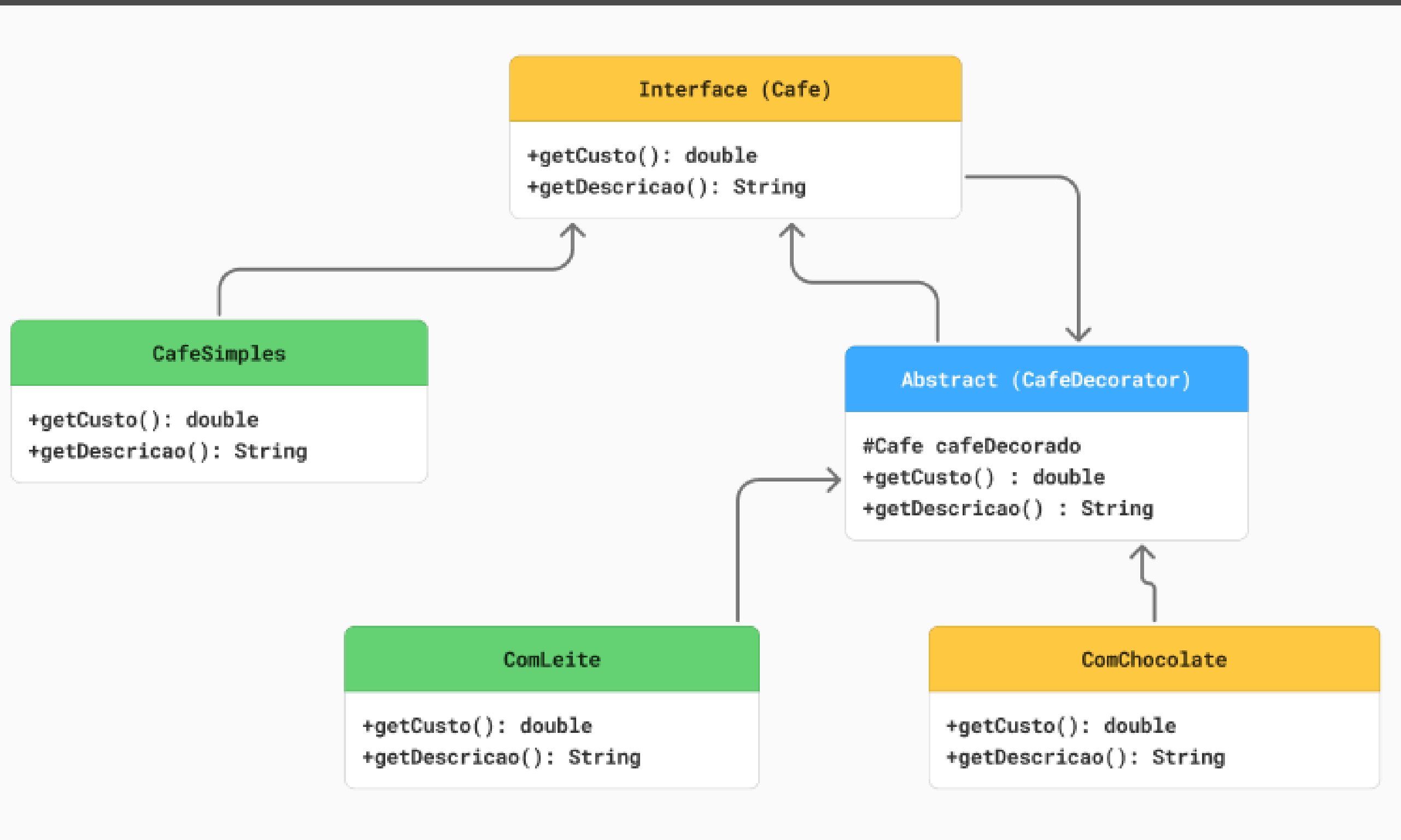
abstract class CafeDecorator implements Cafe {
    protected Cafe cafe;
    public CafeDecorator(Cafe cafe) { this.cafe = cafe; }
}

class ComLeite extends CafeDecorator {
    public ComLeite(Cafe cafe) { super(cafe); }
    public double getCusto() { return cafe.getCusto() + 1.5; }
    public String getDescricao() { return cafe.getDescricao() + " + leite"; }
}

class ComChocolate extends CafeDecorator {
    public ComChocolate(Cafe cafe) { super(cafe); }
    public double getCusto() { return cafe.getCusto() + 2.0; }
    public String getDescricao() { return cafe.getDescricao() + " + chocolate"; }
}

```



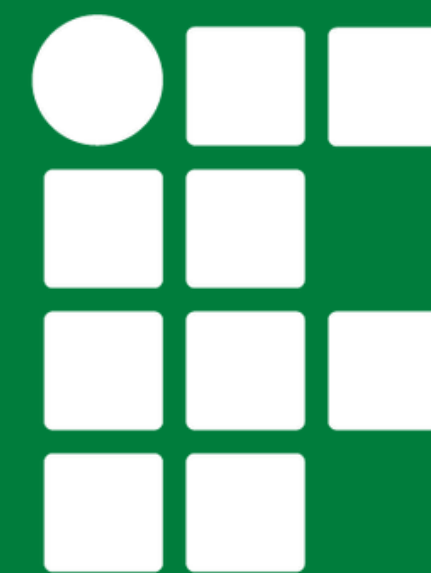


Design Patterns em Delphi 13: Decorator — Flexibilidade Além da Herança. 21 jan. 2026. Disponível em: <https://regys.com.br/design-patterns-em-delphi-13-decorator-flexibilidade-alem-da-heranca-artigo-06/>. Acesso em: 2026.

REFACTORING.GURU. Decorator. Disponível em: <https://refactoring.guru/design-patterns/decorator>. Acesso em: 2026.

ROCHA, Luciano.

Domine Objetos com o Decorator Pattern: Flexibilidade e Reutilização de Código. LinkedIn Pulse. Disponível em: [Design Patterns em Delphi 13: Decorator — Flexibilidade Além da Herança. 21 jan. 2026. Disponível em: https://regys.com.br/design-patterns-em-delphi-13-decorator-flexibilidade-alem-da-heranca-artigo-06/](https://regys.com.br/design-patterns-em-delphi-13-decorator-flexibilidade-alem-da-heranca-artigo-06/). Acesso em: 2026.. Acesso em: 2026.



**INSTITUTO
FEDERAL**

Rio Grande
do Sul

Campus
Farroupilha